

Post-Quantum Web-Evidence Issuance with TLSNotary

How the EU Wallet turns a fact from a real HTTPS session into a verifiable credential: a **TLSNotary** prover co-signs the session with a **post-quantum** notary, the holder chooses exactly **which JSON fields to disclose**, and the issuer mints an SD-JWT web-evidence VC straight into the wallet over OpenID4VCI.

Notary: `notary.euwallet.advatar.systems` · **Issuer:** `vcissuer.advatar.systems` · **Credential:** SD-JWT, `vct dev.advatar.tlsn.evidence.1` · **Transport:** TLS 1.2.

1 · At a glance

TLSNotary lets a holder prove *“this is what the server actually sent me”* without giving the verifier their credentials or a replayable session. The wallet runs the prover against a **notary + proxy**; the notary sees only ciphertext but co-signs a commitment to the transcript. The result is bound into a wallet credential the holder can present later — and the holder decides, field by field, what the credential reveals.

1. Notarise

The prover runs the HTTPS session with a notary that co-signs the transcript — over a **post-quantum** key exchange and a **post-quantum** signature.

2. Disclose

For JSON/API responses the holder picks which leaf fields (RFC 6901 pointers) go into the proof; the rest stay private.

3. Issue

The signed artifact becomes an OpenID4VCI offer; the issuer mints an SD-JWT web-evidence VC into the wallet.

Components

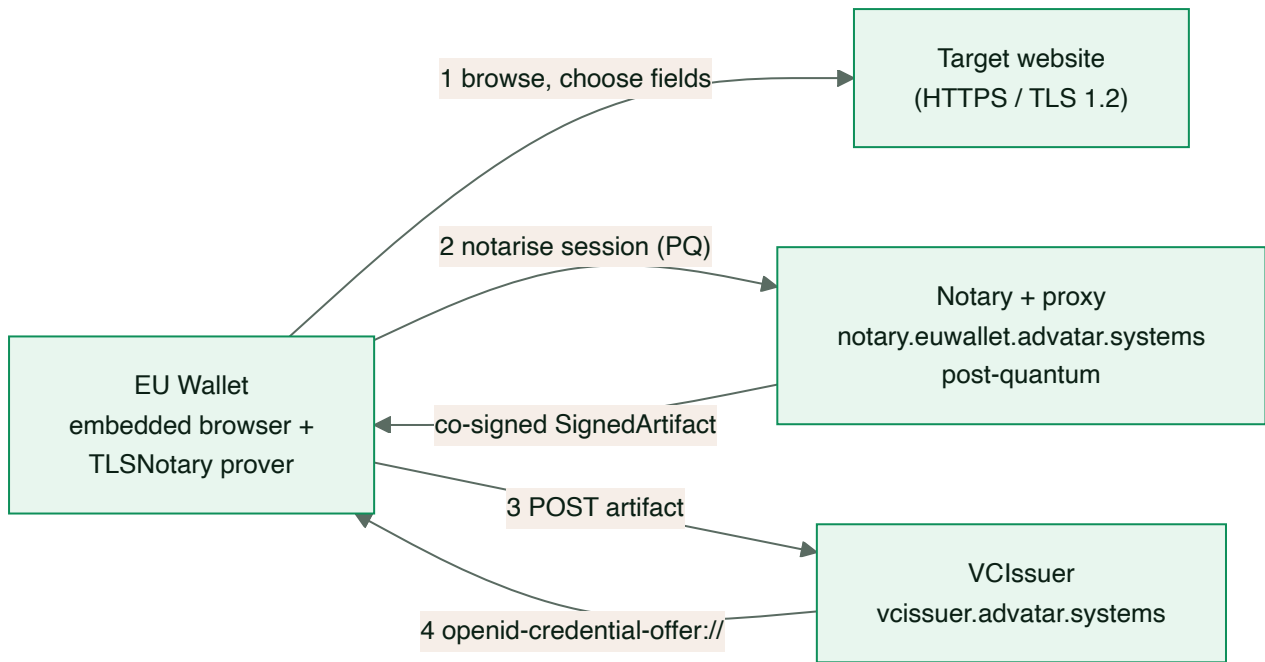


Figure 1. The notary co-signs a commitment to the ciphertext transcript — it never sees plaintext or holder secrets. The wallet carries the artifact to the issuer.

2 · End-to-end sequence

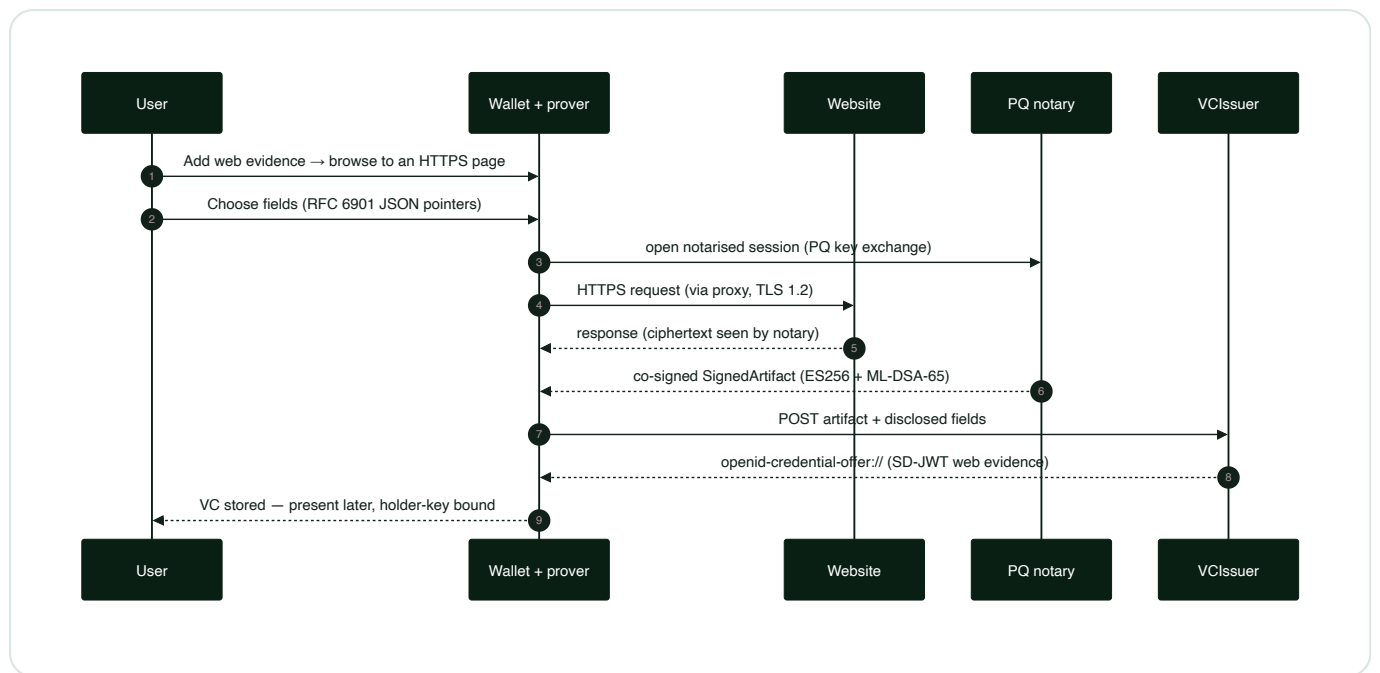


Figure 2. Steps mirror `AddWebEvidenceView` → `TLSNotaryMobileClient.notarize(url, headers, disclosedFields)` → `prepareWalletOffer`. In self-contained demo mode the capture page calls the issuer's `/dev/tlsnotary/demo-offer` (`ENABLE_TLSN_DEMO`); production POSTs to `/evidence-offers/tlsnotary`.

3 · Selective disclosure — the holder chooses what to reveal

REPRESENTATIVE MOCKUP

This is the heart of the flow. When the notarised response is JSON, the wallet flattens it into its leaf fields and lets the holder tick exactly which ones become part of the proof. Everything else is withheld — the verifier learns the disclosed facts and nothing more.

Fields to disclose

the wallet's DisclosureFieldPicker

Reading the screen. The holder is proving their bank account is real and in good standing — without revealing their balance, IBAN, or date of birth.

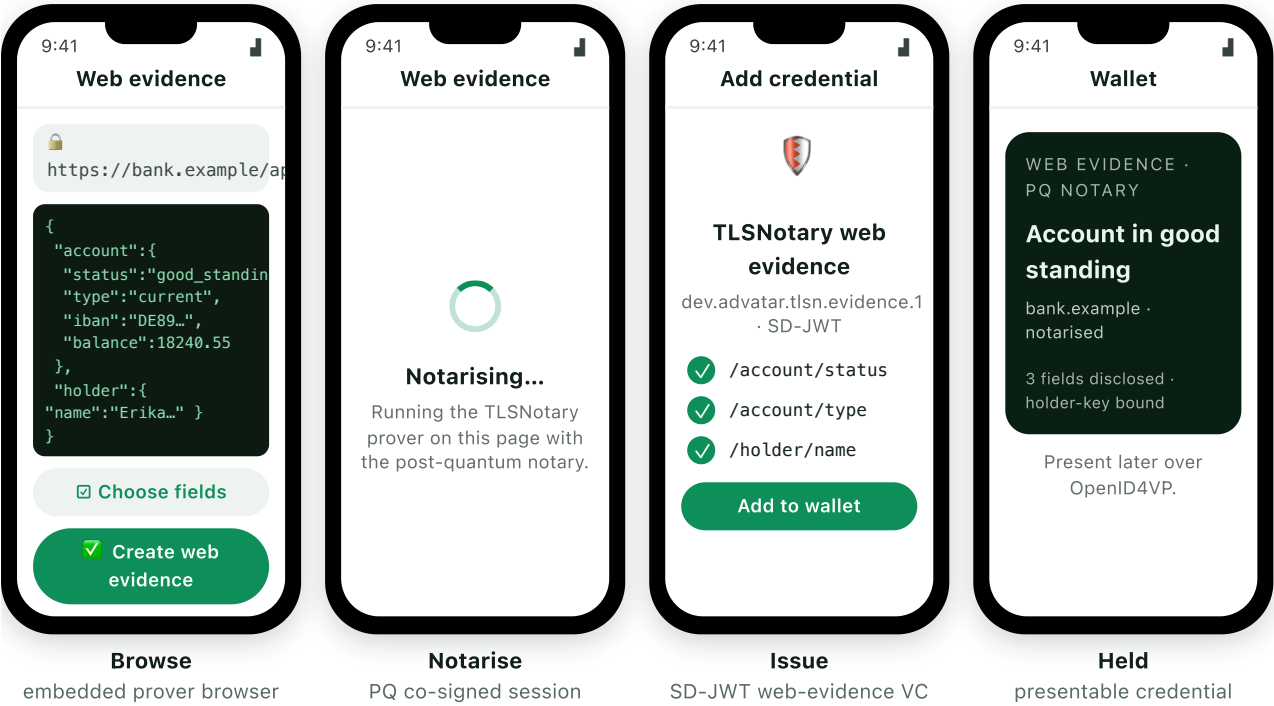
- **Disclosed** — `/account/status` , `/account/type` , `/holder/name`
- **Kept private** — `/account/iban` , `/account/balance` , `/holder/dateOfBirth`

Fields are addressed as **RFC 6901 JSON Pointers**. The default is to disclose everything; the holder *deselects* what to withhold. The chosen pointers are recorded in the credential's `credentialSubject.disclosedFields` .

Honest scope. The picker is the **selection / consent surface**, and the chosen pointers are recorded as disclosed. The transcript-level redaction that reveals *only* the selected byte-spans and cryptographically strips the rest is the remaining core work — until it lands, unselected fields are simply not listed as disclosed rather than being removed from the underlying attestation.

4 • The rest of the flow

REPRESENTATIVE MOCKUPS



Note on the screens. These are faithful **representative mockups** built from the app's real `AddWebEvidenceView` / `DisclosureFieldPicker` copy. The full flow needs a live notary + prover, so it is illustrated rather than captured; send real device screenshots and they can be swapped in.

5 • What makes it post-quantum

A classical TLSNotary proof is only as durable as the classical crypto underneath it — a future quantum adversary who recorded the session could forge or unmask it. This deployment hardens both the key exchange and the notary’s signature with NIST PQC primitives, in **hybrid** mode (a classical algorithm AND a post-quantum one must both hold).

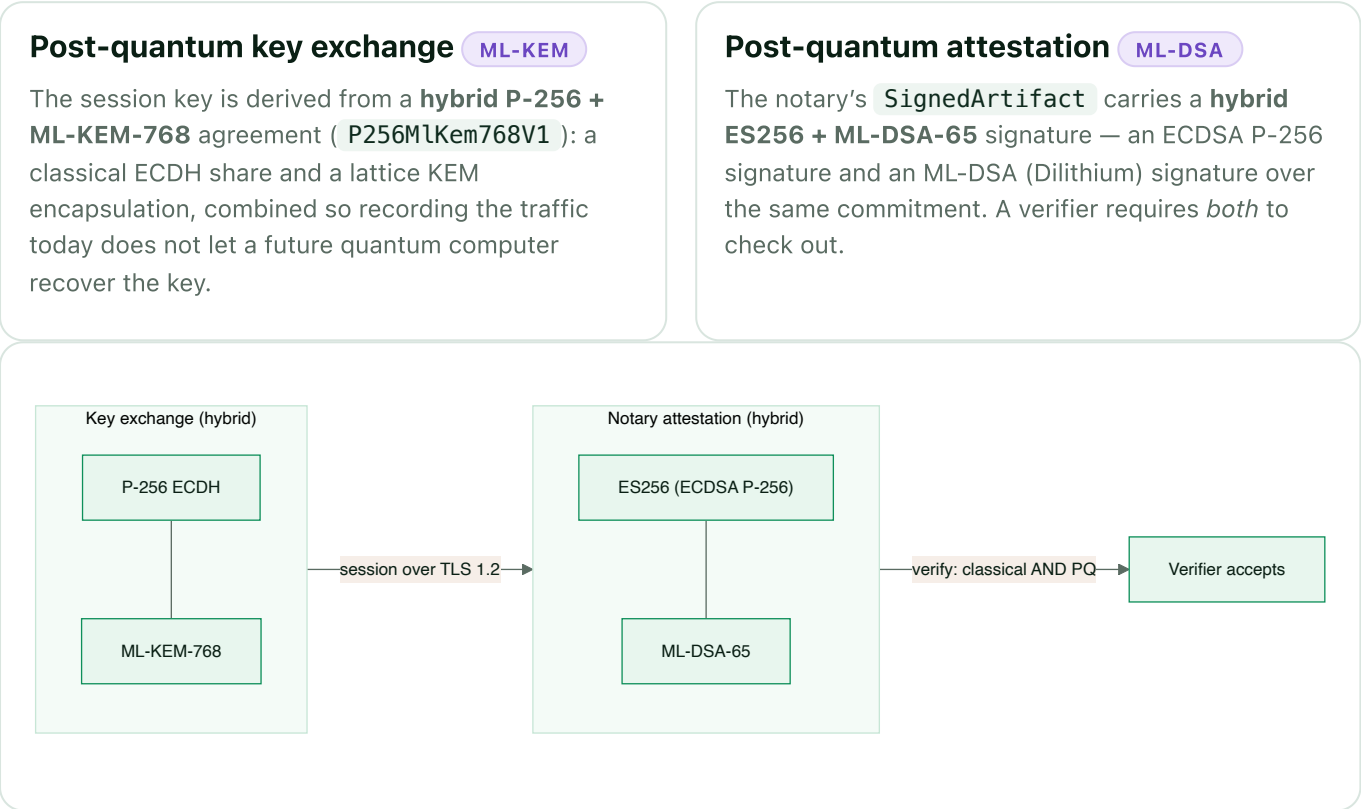


Figure 3. Hybrid everywhere: the proof stays sound as long as *either* the classical or the post-quantum layer is unbroken — so it is safe against both today’s attackers and a future quantum one.

TLSNotary here runs over **TLS 1.2**: the prover pins the session to 1.2 because the notary’s selective-reveal machinery operates on the 1.2 record layer. The post-quantum hardening is layered on top of that transport.

6 · The issued credential & how it verifies

Aspect	Detail
Format	SD-JWT VC, <code>vct dev.advatar.tlsn.evidence.1</code> ("TLSNotary web evidence (development)").
What it asserts	The notarised server + the disclosed JSON fields, listed as RFC 6901 pointers under <code>credentialSubject.disclosedFields</code> , bound to the notary attestation.
Holder binding	Issued to the wallet's key (Secure Enclave when available, software fallback otherwise); presented later over OpenID4VP with proof-of-possession.
Notary trust	The verifier checks the hybrid ES256 + ML-DSA-65 signature against the notary's published key (<code>tlsn.notaryKeyB64u</code>); both components must verify.
Issuance	OpenID4VCI credential offer by value (<code>openid-credential-offer://</code>) handed to the wallet's normal add-credential flow.

Honest limits

- **Selective disclosure** is a consent/selection surface today; transcript-level redaction (reveal only selected spans) is the remaining core work — see §3.
- **Development credential:** the `vct` is dev-scoped; the demo path (`/dev/tlsnotary/demo-offer`) mints a genuine self-signed offer so the wallet loop completes without a live notary. Production uses the WASM prover ↔ notary+proxy and `/evidence-offers/tlsnotary`.
- **TLS 1.2 only** — the notary's selective-reveal works on the 1.2 record layer.

Advatar EU Wallet — Post-Quantum Web-Evidence Issuance with TLSNotary. Notary `notary.euwallet.advatar.systems` · issuer `vcissuer.advatar.systems` · credential `dev.advatar.tlsn.evidence.1` (SD-JWT). PQ: hybrid P-256 + ML-KEM-768 key agreement, hybrid ES256 + ML-DSA-65 attestation. Device screens are representative mockups.